

Northwind Payments — Merchant Console

Architecture notes for `build-battle/merchant-console` , with the payments export traced end to end.

What the app does

An internal console for Northwind's support and ops staff. They use it to look up a payment, refund it, work the dispute queue, and review payouts. The whole thing is fictional and self-contained: every merchant, cardholder, amount, and card number is generated at boot, card numbers use the `4242` test BIN, and nothing persists between restarts.

Its purpose in this repo is as a training target. It was rebranded from Tremor's *Planner* template, and the NOTICE file states that defects were **planted intentionally**. Learners take a ticket (here, NWP-101) end to end and get scored on the PR.

ROUTES

- `/overview` — metrics, monitoring charts, merchant rollup
- `/payments` — filterable, paginated table + Export
- `/payments/[id]` — detail with refunds and dispute
- `/disputes` — queue with evidence deadlines
- `/payouts` — per-merchant settlement batches

API

- `GET /api/payments` — filtered, sorted, paginated JSON
- `GET /api/payments/export` — filtered CSV, fixed columns

That's all. Pages render server-side and read the store directly.

DOMAIN MODEL

- `Merchant` — has an IANA timezone and a currency
- `Payment` — minor-unit amount, status, method, last4
- `Refund` , `Dispute` , `Payout`

Tech stack

LAYER	CHOICE	NOTES
Framework	Next.js 15.1 (App Router), React 19	Server components by default; "use client" only where there's interaction (filter bar, charts, sidebar)
Language	TypeScript 5.9	@/ path alias → src/
UI	Tailwind 3.4, Radix primitives, Tremor-style components	Tailwind only — no inline styles or CSS modules. Icons from lucide and remixicon; charts via Recharts
Theming	next-themes	Class-based dark mode, system default

Data	In-memory store on <code>globalThis</code>	Deterministic PRNG (seed <code>20260813</code>), 120 days × 14 payments/day, anchored at <code>2026-08-13Z</code> . No DB, no ORM — persistence is NWP-203
Tests	Vitest 3, node env, no DOM	Covers <code>src/lib/{money,dates,csv}</code> only. Runs in under a second
Tooling	ESLint (next), Prettier + tailwind plugin	

How `src/` is organized

<code>src/</code>	
├─ app/	Next.js App Router
│ ├─ layout.tsx	Sidebar + breadcrumbs shell, ThemeProvider, Geist fonts
│ ├─ page.tsx	redirect → /overview
│ ├─ siteConfig.ts	name + base links
│ ├─ api/payments/	
│ │ ├─ route.ts	GET /api/payments → parseFilters + queryPayments
│ │ └─ export/route.ts	GET /api/payments/export → CSV
│ ├─ overview/	metrics page, monitoring charts, merchants tab
│ ├─ payments/	
│ │ ├─ page.tsx	server component: reads store, renders table, Export link
│ │ ├─ filter-bar.tsx	client component: search / status / merchant → URL params
│ │ └─ [id]/page.tsx	payment detail
│ ├─ disputes/page.tsx	
│ └─ payouts/page.tsx	
├─ components/	Tremor/Radix primitives (Button, Input, Select, Table, Drawer...)
│ └─ ui/	console-specific: AppSidebar, Breadcrumbs, StatusBadge, MetricsCards
├─ data/	the "database"
│ ├─ types.ts	Payment, Merchant, PaymentFilters, ...
│ ├─ merchants.ts	8 fictional merchants, US/EU/UK timezones
│ ├─ generate.ts	deterministic seed generator
│ ├─ store.ts	globalThis-held singleton built from generate()
│ ├─ queries.ts	parseFilters · filterPayments · sortPayments · paginate · queryPayment
│ ├─ metrics.ts	headline numbers, daily volume
│ └─ analytics.ts	weekly buckets, dispute rate, merchant rollup
└─ lib/	pure helpers, each with a .test.ts beside it
│ ├─ money.ts	formatMoney, sumMinorUnits, parseAmountToMinorUnits
│ ├─ dates.ts	utcDayKey, dayKeyInZone, formatInZone, formatDate, ...
│ ├─ csv.ts	EXPORT_COLUMNS, toCsv, exportFilename
│ └─ utils.ts, use*.ts	cx() and small hooks

Dependency direction: `app` → `data` → `lib` , mostly. One inversion worth knowing: `lib/csv.ts` imports `merchantById` from `data/merchants` to resolve merchant names, so the CSV helper isn't quite pure.

Context layering

Rules are loaded by path, not all at once. `CLAUDE.md` holds the four core conventions; `.claude/rules/money.md` , `api-routes.md` , `cards.md` , and `components.md` attach to matching files.

- **Money is integer minor units.** `$250.00` is `25000` . Format once, at the edge, next to its currency code.
- **Storage and bucketing are UTC.** Only display converts to the merchant's timezone.
- **One query builder.** All payment filtering goes through `data/queries.ts` . A second one is a defect.
- **Validate on the server.** Client input is checked against an allowlist before it reaches a query, a filename, or the store.

Payments export, end to end

This is the flow NWP-101 changes. Today it is a plain anchor tag that hits a route handler; no dialog, no column choice, no scope choice.

1

`src/app/payments/filter-bar.tsx` **client**

Ops sets filters

Search, status, and merchant selects call `router.push("/payments?status=...&merchantId=...&search=...")` . Filter state lives entirely in the URL — there is no client store.

2

`src/app/payments/page.tsx` **server**

Page renders the table and the Export link

Reads `searchParams` , builds a `PaymentFilters` by hand (note: it duplicates the status allowlist rather than calling `parseFilters`), calls `queryPayments` for the 20-row page, and renders `<a href="/api/payments/export?{same query}">Export</a>` . The export button forwards whatever is in the URL, including `page` .

3

`src/app/api/payments/export/route.ts` **server**

Route handler parses and queries

`parseFilters(searchParams)` allowlists status / sort / direction and passes merchantId, search, from, to through. Then `sortPayments(filterPayments(filters))` — deliberately *not* `queryPayments` , so it skips `paginate` and exports every matching row, not just the current page.

4

`src/data/queries.ts` **server**

The one query builder does the work

`filterPayments` scans `store.payments` in memory: status, merchant, ISO-string date range, and a case-insensitive search across id, description, last4, and merchant name. `sortPayments` orders by `createdAt` or `amount` .

5

`src/lib/csv.ts` **server**

Rows become CSV

`toCsv(rows, columns = EXPORT_COLUMNS)` already accepts a column list — the caller just never passes one. Each cell is resolved by a `switch` over `ExportColumn` ; `amount` is formatted via `formatMoney` (symbol + separators, hence quoted), and `currency` is its own column. Cells with commas, quotes, or newlines are RFC-4180 escaped. `exportFilename()` returns `payments-YYYY-MM-DD.csv` in UTC.

6

Response

Browser downloads the file

`text/csv; charset=utf-8` with `content-disposition: attachment` . The browser's native download handles it; nothing client-side touches the bytes.

What already exists vs. what NWP-101 adds

ACCEPTANCE CRITERION	TODAY	GAP
Choose columns, last4 off by default	<code>toCsv</code> takes a column list; <code>EXPORT_COLUMNS</code> is the allowlist	No UI, route never reads a <code>columns</code> param, no server-side validation of one
Scope: current filter vs. all	Route already honors filters and ignores pagination	No way to ask for "all"; no row count shown before download
Filename reflects scope + date	<code>exportFilename(date)</code> → <code>payments-2026-08-13.csv</code>	Needs a scope segment, e.g. <code>payments-disputed-...</code>
Minor units internally, currency column	Already true in <code>csv.ts</code>	Keep it that way
No columns → Download disabled	—	Client-side gate in the new dialog; server should still reject an empty list

Things I noticed while reading. Not a review — just landmarks for the ticket.

- `sortPayments` sorts by `amount` using `String(a.amount).localeCompare(...)`, i.e. lexically. `999` sorts above `10000`. The comment claims this matches the table, but the table never sorts by amount. Looks planted.
- `payments/page.tsx` re-implements status parsing instead of calling `parseFilters`, so the page and the API can drift. Convention #3 in spirit.
- `csv.test.ts` already covers escaping and column selection. The ticket says extend that file, not start a new one.
- The existing Export anchor is a plain `<a>`; the new dialog will need the Radix `Dialog` dependency that's already installed (`@radix-ui/react-dialog`) but has no wrapper in `src/components/` yet — `Drawer.tsx` is the closest existing pattern.